

Informe técnico — Selección y ordenación de fragmentos de video educativo

Proyecto Final IA · Tema 2

1. Problema inicial

1.1. Contexto

Un video educativo largo (charla TED, clase grabada, etc.) se divide en **fragmentos** con transcripción y duración. El usuario necesita un **resumen en video** — un subconjunto de esos fragmentos reproducidos en un orden que conserve el sentido pedagógico — pero con un **límite de duración** (p.ej. 45 s para un teaser, o el 25 % del material total).

1.2. Formulación

Dado un conjunto de fragmentos $s_1, \dots, s_n$, cada uno con:

- texto transcrito,
- duración l_i ,
- posición temporal en el video original (`start_time`, `end_time`),

se busca una **secuencia ordenada** de índices $\pi = (j_1, j_2, \dots, j_k)$ tal que:

1. **Restricción de capacidad:** $\sum_{t=1}^k l_{j_t} \leq L_{\text{máx}}$ (knapsack con límite de tiempo).
2. **Objetivo:** maximizar una función de **relevancia** (¿aporta contenido educativo?) y **coherencia** (¿encaja narrativamente con el fragmento anterior en el resumen?).

El enunciado del Tema 2 exige un componente **algorítmico clásico** y el uso del LLM como **evaluador semántico** (relevancia y coherencia), no como generador único del resumen ni como decisor directo de qué fragmentos incluir. La selección y el orden los resuelve un **solver de optimización** (programación dinámica o heurística) que consume las puntuaciones del LLM. Detalle en la Sección ??.

1.3. Por qué no basta un knapsack estándar

Un knapsack clásico elige objetos sin orden. Aquí el orden importa: la coherencia se define entre **pares consecutivos en la secuencia de reproducción**, no entre vecinos del array de entrada. Además, el input puede llegar **desordenado** en el JSON (p.ej. metadatos permutados); el sistema debe **reconstruir un orden narrativo**, no solo filtrar fragmentos.

Implementación formal: `src/problem.py` (`SelectionProblem`, `Fragment`).

2. Algoritmos implementados y funcionamiento

2.1. Baseline clásico (solve_baseline)

Idea: referencia sin LLM. Todos los fragmentos tienen la misma relevancia (1.0); la coherencia entre pares se aproxima con la **proximidad temporal** (`start_time`).

Enrutado automático (Task E):

n fragmentos	Modo	Algoritmo
≤ 12	<code>exact_reorder</code>	DP con bitmask (<code>src/solver/reorder.py</code>)
> 12	<code>heuristic_reorder</code>	Subconjunto greedy + inserción greedy por coherencia

El baseline demuestra que existe un **solver algorítmico real** independiente del LLM y sirve de contraste en los experimentos.

2.2. DP con orden fijo — subsecuencia (ordered_knapsack_dp)

Cuándo: funciones legacy `solve_with_llm`, `solve_with_llm_static`; modo `experimental-static`.

Estado: (índice, duración_restante) o (índice, duración_restante, último_seleccionado) si hay coherencia dinámica.

Transición: en cada posición del array de entrada, **incluir u omitir** el fragmento. La salida es siempre una **subsecuencia** (índices crecientes): se respeta el orden del JSON.

Coherencia dinámica (`ordered_knapsack_dp_with_coherence`): el LLM puntúa coherencia entre pares (último, j) bajo demanda; el DP, al incluir el fragmento j , suma $\text{relevancia}[j] + w \times \text{coherencia}(\text{último}, j)$. Permite **saltar** fragmentos intermedios manteniendo coherencia entre no vecinos.

Coherencia estática (`solve_with_llm_static`): el LLM puntúa coherencia solo entre vecinos ($i, i+1$) del input; esos valores se combinan con relevancia **antes** del DP. Falla en escenarios donde hay que saltar un fragmento puente (`bench_static_vs_dynamic.json`).

Código: `src/solver/baseline.py`, `src/solver/llm_assisted.py`.

2.3. DP con reordenación exacta — bitmask (unordered_knapsack_dp_with_coherence)

Cuándo: `solve()` / `solve_baseline()` con $n \leq 12$; flujo de producción en instancias `bench`.

Estado: (`mask`, `last`, `remaining`) — `mask` indica qué fragmentos ya están en el resumen; `last` es el último añadido.

Transición: añadir cualquier j no incluido en `mask` si cabe en duración; el **DP** maximiza relevancia + coherencia entre pares consecutivos **en la permutación elegida** (pesos provistos por el LLM).

Complejidad: $O(2^n \cdot n \cdot \text{capacidad})$. Óptimo para n pequeño (`bench`: $n = 5$; mini-videos: $n = 10$).

Matriz de coherencia: el **LLM** puntúa $\text{coherencia}[i, j]$ para todo par (i, j) — $O(n^2)$ llamadas, cacheadas en `.llm_cache.json`; el **DP** consume esa matriz al explorar permutaciones.

Código: `src/solver/reorder.py`.

2.4. Heurística en dos fases ($n > 12$)

Cuándo: videos completos (`video_*.json`, 25–50 fragmentos).

1. `select_subset_greedy`: elige fragmentos por ratio relevancia/duración hasta llenar `max_duration`.
2. `order_subset_greedy`: inserta cada fragmento restante en la posición que maximiza relevancia + coherencia local.

Complejidad: $O(n^2)$ por fase — escalable, pero **no garantiza optimalidad global**.
Código: `src/solver/reorder.py` (`heuristic_reorder_with_coherence`).

2.5. Rol del LLM — qué hace y qué no hace

El LLM es una **función de evaluación semántica**. No implementa el knapsack ni devuelve la lista final de fragmentos: devuelve **números** que el solver clásico usa como entrada.

2.5.1. División de responsabilidades

Componente	Responsabilidad
LLM (<code>score_fragment</code> , <code>score_coherence</code>)	Puntuar relevancia de cada texto y coherencia entre dos textos
Solver clásico (DP, heurística)	Decidir qué fragmentos entran y en qué orden , respetando max
Baseline (<code>solve_baseline</code>)	Misma optimización con relevancia uniforme y coherencia proxy
Evaluación post-hoc (<code>-evaluate</code>)	Medir con el mismo LLM la calidad de cualquier selección ya p

2.5.2. Qué hace el LLM

1. **Durante la resolución** (`solvers llm_dynamic`, `llm_static`): por cada fragmento llama a `score_fragment(texto)`; para coherencia llama a `score_coherence(texto_a, texto_b)` entre pares relevantes (vecinos consecutivos en modo estático, o cualquier par en modo dinámico/reordenación). Esas puntuaciones son los **pesos** de la función objetivo del DP.
2. **Durante la evaluación experimental** (`evaluate_selection_with_llm`): vuelve a puntuar relevancia y coherencia sobre la selección **ya generada** por cada solver, para calcular `mean_relevance`, `mean_coherence` y `objective_score` con criterio homogéneo. Así el baseline también se juzga con el mismo estándar semántico.

Prompts (solo piden un número, no una selección): `src/llm/prompts.py`.

2.5.3. Qué no hace el LLM

- **No genera** el texto del resumen ni edita vídeo.
- **No devuelve** directamente índices, permutaciones ni listas de fragmentos.
- **No sustituye** el algoritmo de optimización: sin DP/heurística, las puntuaciones no producen una selección válida bajo el límite de duración.
- **No impone** restricciones duras (duración, cardinalidad); eso lo garantiza solo el solver.

En resumen: el LLM responde preguntas del tipo «¿qué tan relevante es este fragmento?» y «¿encajan estos dos fragmentos seguidos?»; el **algoritmo** resuelve el problema combinatorio de knapsack con orden.

2.5.4. Flujo con LLM (`solver llm_dynamic` / `solve()`)

```
Fragmentos + max_duration
|
v
LLM -> relevancia[i] para cada fragmento
LLM -> coherencia[i,j] para pares (segun modo)
|
v
DP bitmask o heuristica -> maximiza sum relevancia + w*sum coherencia
                        sujeto a sum duracion <= max_duration
```

Salida: lista ordenada de índices (decision del algoritmo)

Las respuestas LLM se cachean en `.llm_cache.json` para reproducibilidad y ahorro de API.
 Código: `src/llm/client.py`, `src/llm/prompts.py`, `src/llm/cache.py`, `src/experiments/metrics.py`.

3. Por qué estos algoritmos encajan en el problema

Requisito del problema	Algoritmo que lo cubre
Límite de duración (knapsack)	DP / heurística (restricción hard, sin LLM)
Maximizar relevancia semántica	LLM puntúa; DP/heurística optimiza con esos pesos
Coherencia entre fragmentos consecutivos del resumen	LLM puntúa pares; DP suma coherencia en transición
Orden del video original o reconstruido	DP (subsecuencia o bitmask); no lo decide el LLM
Componente algorítmico clásico visible	Baseline + DP siempre presentes; LLM solo evalúa
Escalabilidad a videos largos	Heurística $n > 12$; LLM sigue siendo $O(n^2)$ en coherencia

La **programación dinámica** es la herramienta natural para knapsack con decisiones secuenciales. La extensión con **bitmask** generaliza a permutaciones cuando el input no está ordenado. La **heurística** es el compromiso estándar cuando la optimalidad exacta es computacionalmente inviable.

4. Algoritmos testeados en la suite lite (`execute.md`)

Corrida completada el 2026-06-11 (proveedor Groq, modelo llama-3.3-70b-versatile). Detalle por bloque en `informe/notas_experimentos.md`.

Solver (CSV)	Función	Papel del LLM	Papel
<code>baseline</code>	<code>solve_baseline()</code>	Ninguno en resolución	DP
<code>llm_dynamic</code>	<code>solve()</code>	Puntúa relevancia y coherencia (todos los pares necesarios)	DP
<code>llm_static</code>	<code>solve_with_llm_static()</code>	Puntúa relevancia y coherencia solo entre vecinos ($i, i + 1$)	DP

En los tres casos, la **evaluación post-hoc** (`-evaluate`) usa el mismo LLM como juez externo para medir `objective_score`, también en filas `baseline`.

Instancias ($5 \times 3 = 15$ filas):

Instancia	Objetivo del test
<code>example_instance.json</code>	Caso base: todos caben en el límite
<code>example_instance_overlimit.json</code>	Recorte bajo límite estricto (<code>max_duration=30</code>)
<code>bench_static_vs_dynamic.json</code>	Salto narrativo: estático vs dinámico
<code>bench_irrelevant_middle.json</code>	Omisión de fragmento irrelevante (música/anuncio)
<code>bench_disordered.json</code>	Input permutado $\rightarrow$ reordenación hacia orden narrativo

En todas las instancias bench, $n = 5 \rightarrow$ modo `exact_reorder` (DP bitmask óptimo).

Artefactos: `results/experiments_bench.csv`, `results/summary_bench.md`, `results/comparison_bench`

5. Distinción por tamaño de instancia ($n \leq 12$ vs $n > 12$)

El solver unificado (`solve`, `solve_baseline`) elige la estrategia según `REORDER_EXACT_LIMIT = 12`:

<code>n <= 12</code>	<code>-> exact_reorder</code>	(DP bitmask, óptimo)
<code>n > 12</code>	<code>-> heuristic_reorder</code>	(greedy, aproximado)

5.1. Motivo

La reordenación óptima con bitmask explora hasta 2^n estados. Con $n = 50$ hay del orden de 10^{15} configuraciones — **imposible** en tiempo razonable. Con $n = 5$ (bench) son 32 — **demonstrable y reproducible** en el informe.

5.2. Consecuencias de no distinguir

Si se usa...	En videos largos	En bench ($n = 5$)
Solo DP exacto	Timeout / memoria	Correcto
Solo heurística	Escalable	Subóptimo; pierde rigor experimental
Solo subsecuencia (sin reordenar)	Input ordenado OK	bench_disordered falla — resumen incoherente

El umbral 12 permite optimalidad en `mini_video_*.json` ($n = 10$) y heurística en `video_*.json` ($n \approx 25$ – 50). Los experimentos cuantitativos del informe se limitaron a $n = 5$ por **coste de API** ($O(n^2)$ llamadas de coherencia), no por limitación algorítmica.

Documentación ampliada: `README.md` (sección *Por qué se distingue el tamaño*), `planning/walkthroughE.md`.

6. Resultados y análisis

6.1. Metodología de la corrida

Se ejecutó la suite lite descrita en la Sección 4 siguiendo el plan manual `execute.md` (bloques 3–6b, fusión y Fase 8). Condiciones:

- **Proveedor:** Groq · **Modelo:** llama-3.3-70b-versatile
- **Caché LLM** borrada al inicio para scores frescos
- **Tres solvers por instancia:** `baseline` (sin LLM al resolver), `llm_dynamic` (LLM puntúa → DP reordena), `llm_static` (LLM puntúa vecinos → DP en subsecuencia fija)
- **Evaluación post-hoc** (`-evaluate`): el LLM actúa aquí como **juez imparcial** — repuntúa relevancia y coherencia de la selección ya producida por cada solver (incluido el `baseline`) y calcula `objective_score` con la misma fórmula y peso 0,25 sobre coherencia. Así se separa *cómo se generó* la solución de *cómo se mide* su calidad semántica.
- **15 filas** en `results/experiments_bench.csv`; tabla agregada en `results/summary_bench.md`; gráfico en `results/comparison_bench.png`

En todas las instancias, $n = 5 \rightarrow$ modo `exact_reorder` (DP bitmask óptimo).

6.2. Panorama global

Instancia	Mejor <code>objective_score</code>	Ganador	vs baseline
<code>example_instance</code>	3,625	<code>llm_dynamic</code>	+5,8 %
<code>example_instance_overlimit</code>	2,325	<code>llm_dynamic</code>	+17,7 %
<code>bench_static_vs_dynamic</code>	1,550	<code>llm_dynamic</code> / <code>llm_static</code> (empate)	+51,2 %
<code>bench_irrelevant_middle</code>	0,675	<code>llm_dynamic</code>	+12,5 %
<code>bench_disordered</code>	3,175	<code>llm_dynamic</code>	+49,4 %

`llm_dynamic` **obtiene el mejor `objective_score` en las cinco instancias**. En `bench_static_vs_dynamic` coincide con `llm_static` porque, al saltar el fragmento puente, solo queda una transición y ambos enfoques LLM eligen la misma pareja [0, 2].

A nivel agregado, el baseline queda sistemáticamente por debajo en calidad semántica post-hoc: **no usa el LLM al resolver** (relevancia uniforme 1,0 y coherencia proxy por `start_time`), por lo que el DP maximiza **cobertura de duración** más que **contenido educativo**. En cuatro de cinco casos llena o casi llena el cupo temporal sin discriminar fragmentos de baja relevancia. Los solvers asistidos por LLM obtienen mejores `objective_score` porque el **mismo criterio semántico** que evalúa al final también guió (vía puntuaciones) la optimización del DP.

6.3. Análisis por instancia

6.3.1. Caso base — `example_instance.json` (`max_duration = 45 s`)

Los tres solvers seleccionan los **cinco fragmentos** (utilización 100 %). La diferencia está en el **orden de reproducción**:

Solver	Selección	<code>objective_score</code>	<code>mean_coherence</code>
<code>baseline</code>	[0, 1, 2, 3, 4]	3,425	0,500
<code>llm_static</code>	[0, 1, 2, 3, 4]	3,425	0,500
<code>llm_dynamic</code>	[0, 2, 1, 3, 4]	3,625	0,700

Cuando caben todos los fragmentos, el **DP asistido por LLM** no elimina contenido: usa las puntuaciones de coherencia entre pares para **reordenar** (p. ej. intercambia 1 y 2 $\rightarrow$ [0, 2, 1, 3, 4]). El LLM no devuelve ese orden; lo encuentra el DP bitmask al maximizar relevancia + coherencia. El baseline y el estático, sin matriz completa de coherencia y atados al orden del JSON, no explotan esa mejora. Es el efecto más sutil de la suite: la ganancia viene del orden decidido por el algoritmo, no del subconjunto.

6.3.2. Límite estricto — `example_instance_overlimit.json` (`max_duration = 30 s`)

Con el mismo contenido pero menos tiempo, hay que **recortar** a tres fragmentos (utilización ~90 %):

Solver	Selección	<code>objective_score</code>	<code>mean_relevance</code>
<code>baseline</code>	[2, 3, 4]	1,975	0,767
<code>llm_static</code>	[0, 2, 4]	2,200	0,867
<code>llm_dynamic</code>	[0, 2, 1]	2,325	0,833

El baseline, sin señal semántica del LLM, conserva el **final** del array (índices 2–4). Los solvers asistidos usan **scores de relevancia del LLM como pesos del DP**, que priorizan fragmentos con mayor puntuación — incluyendo la introducción (índice 0) —; el dinámico además optimiza

la coherencia entre las transiciones [0, 2, 1]. La brecha de +17,7 % frente al baseline confirma que, bajo presión de capacidad, las puntuaciones LLM cambian **qué** entra en el resumen; la decisión final sigue siendo del algoritmo.

6.3.3. Salto narrativo — bench_static_vs_dynamic.json

Escenario diseñado: coherencia alta entre fragmentos 0 y 2, baja en el puente 1. Límite ajustado para forzar exactamente dos fragmentos.

Solver	Selección	objective_score
baseline	[0, 1]	1,025
llm_static	[0, 2]	1,550
llm_dynamic	[0, 2]	1,550

El baseline elige el par consecutivo [0, 1] por proximidad temporal (sin LLM). En los solvers asistidos, el **DP consume scores de coherencia del LLM** y **salta** el fragmento puente 1, enlazando $0 \rightarrow 2$ — alineado con el diseño del bench. En esta corrida **estático y dinámico convergen**: al quedar solo dos fragmentos hay una única transición, así que precalcular coherencia entre vecinos ($i, i + 1$) basta. La lección del caso no es la divergencia estático/dinámico, sino que **puntuar coherencia semánticamente** (LLM) + **optimizar** (DP) supera al criterio temporal del baseline (+51 %).

6.3.4. Fragmento irrelevante — bench_irrelevant_middle.json

Fragmento 2 (música/anuncio) en el centro; límite que impide incluir todo el material útil.

Solver	Selección	¿Incluye idx 2?	objective_score
baseline	[0]	No	0,600
llm_static	[0]	No	0,600
llm_dynamic	[1]	No	0,675

En el resultado final, los tres solvers **excluyen el índice 2** del resumen, pero por motivos distintos:

- **Baseline:** el DP sin LLM elige un solo fragmento [0] por la combinación de duraciones y scores uniformes; **no** identifica el índice 2 como «música/anuncio».
- **LLM estático / dinámico:** el LLM asigna **baja relevancia** al fragmento irrelevante; el DP, al optimizar con esos pesos, lo deja fuera de forma intencional.

El dinámico elige el índice 1 frente al 0 del estático (relevancia 0,90 vs 0,80) y aprovecha mejor el cupo (93 % vs 74 %). Con un solo fragmento la coherencia media es 0 en todos los casos; la diferencia es de **relevancia** (puntuada por el LLM) y duración (decidida por el DP).

6.3.5. Input permutado — bench_disordered.json

Mismos textos que el caso anterior, pero el array JSON está **desordenado**. Comprueba reordenación automática (Task E).

Solver	Selección	¿Incluye idx 2?	mean_coherence	objective_score
baseline	[1, 4, 2, 3]	Sí	0,333	2,125
llm_static	[0, 1, 3, 4]	No	0,600	3,000
llm_dynamic	[1, 4, 0, 3]	No	0,833	3,175

Es el contraste más contundente de la suite:

1. **Baseline sin LLM:** incluye el fragmento irrelevante (índice 2) en `[1, 4, 2, 3]` porque el DP solo ve duraciones y coherencia proxy temporal — no distingue ruido de contenido educativo. Coherencia media post-hoc: 0,33.
2. **LLM estático:** el LLM penaliza el fragmento 2; el DP omite el irrelevante y devuelve subsecuencia `[0, 1, 3, 4]`, limitada al orden del JSON permutado.
3. **LLM dinámico:** mismas puntuaciones semánticas, pero el **DP bitmask** también **reordena** $\rightarrow [1, 4, 0, 3]$ (introducción $\rightarrow$ definición $\rightarrow$ ejemplo $\rightarrow$ conclusión), con coherencia media 0,83 y **objective_score** un **49 % superior** al baseline.

El LLM aporta la **semántica**; el algoritmo aporta la **decisión combinatoria** (inclusión, exclusión y permutación). Este bloque valida el requisito del Tema 2 de **seleccionar y ordenar**.

6.4. Comparación estático vs dinámico

Instancia	¿Coinciden estático y dinámico?	Interpretación
<code>example_instance</code>	No (solo orden)	Dinámico reordena; estático sigue el JSON
<code>example_instance_overlimit</code>	No (<code>[0, 2, 4]</code> vs <code>[0, 2, 1]</code>)	Dinámico optimiza transiciones al recortar
<code>bench_static_vs_dynamic</code>	Sí (<code>[0, 2]</code>)	Una sola transición; ambos aciertan el salt
<code>bench_irrelevant_middle</code>	No (<code>[0]</code> vs <code>[1]</code>)	Dinámico elige fragmento más relevante
<code>bench_disordered</code>	No (órdenes distintos)	Dinámico explota matriz completa de cohe

El enfoque **estático** es útil como baseline experimental legacy (LLM puntúa menos pares; DP restringido a subsecuencia), pero en escenarios de reordenación o recorte agresivo el **dinámico** (`solve()`: LLM puntúa todos los pares necesarios + DP bitmask) domina o iguala en todos los casos. La divergencia es mayor cuando importa la coherencia entre fragmentos **no consecutivos en el input** — precisamente el caso de uso del proyecto.

6.5. Lectura de las métricas

- **solver_score:** valor interno que el **algoritmo** maximizó al resolver (relevancia uniforme en baseline; suma de scores LLM + coherencia en solvers asistidos). No es comparable entre solvers distintos porque cada uno optimiza una función distinta.
- **objective_score (post-hoc):** métrica **común** calculada **después** por el mismo LLM evaluador sobre la selección ya generada. Separa el rol del LLM-juez del rol del LLM-puntuador-durante-resolución. Es la base justa para comparar filas del CSV.
- **duration_utilization:** fracción del límite temporal usada (decisión del DP). Un baseline alto no implica buen resumen si llena con fragmentos irrelevantes (`bench_disordered`: 100 % utilización pero baja calidad semántica).
- **mean_relevance / mean_coherence:** descomposición del **objective_score**; reflejan puntuaciones LLM sobre el resumen final, no decisiones del algoritmo.

6.6. Conclusiones de la evaluación

1. **El LLM como evaluador aporta valor medible:** al **puntuar** relevancia y coherencia (no al elegir índices), habilita un DP que maximiza calidad semántica bajo límite de duración. `llm_dynamic` obtiene el mejor **objective_score** post-hoc en las cinco instancias (+5,8 % a +49,4 % vs baseline).

2. **El baseline cumple su rol de contraste algorítmico:** resuelve el knapsack **sin LLM en resolución**; demuestra qué aporta la capa semántica cuando el mismo LLM juzga después a todos por igual.
3. **La reordenación automática es del algoritmo, no del LLM:** `bench_disordered` muestra que, sin scores semánticos, el DP baseline incluye ruido; con scores LLM + DP bitmask, la selección y el orden narrativo mejoran.
4. **Coherencia dinámica vs estática:** equivalentes cuando la selección reduce el resumen a un salto simple; el dinámico gana cuando el DP debe considerar pares no vecinos en el JSON.
5. **Limitación de alcance:** resultados en instancias sintéticas de 5 fragmentos con un único modelo (Groq). El rol del LLM (puntuar) y del algoritmo (optimizar) se mantiene en $n > 12$, pero la heurística sustituye al DP exacto.

Gráfico comparativo de `objective_score` por instancia y solver: `results/comparison_bench.png`.

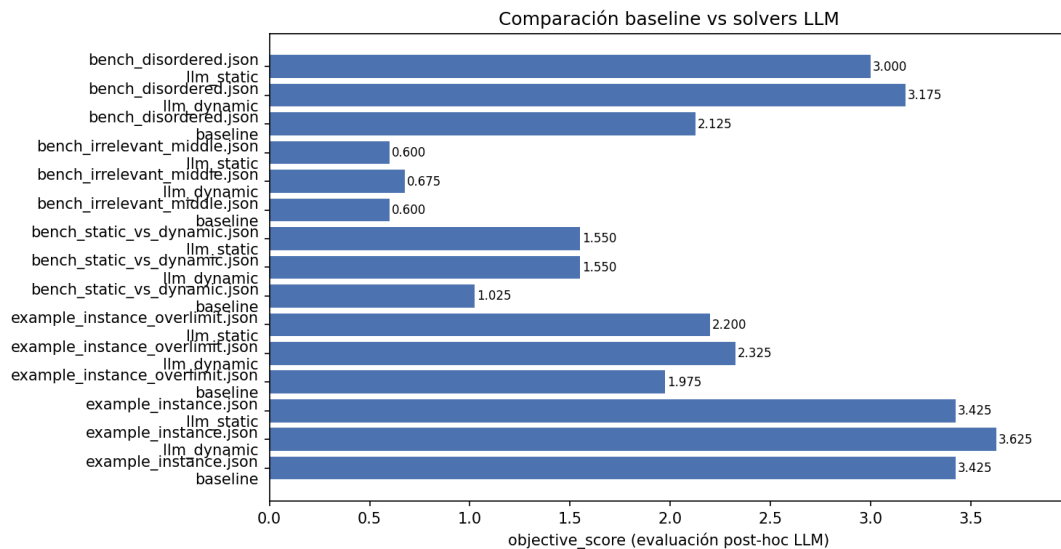


Figura 1: Comparación de `objective_score` por instancia y solver.

7. Limitaciones y trabajo pendiente

- **Coste API:** no se ejecutaron `mini_video_*` ni `video_*` con LLM; la suite lite (sintéticas, $n = 5$) es la base cuantitativa.
- **Heurística $n > 12$:** no evaluada experimentalmente con LLM real en este informe; validada con tests mock ($n = 15$).
- **Modelado formal / capítulos temas-simulacion.pdf:** pendiente de redactar (lógica difusa para grados de relevancia, agente percibe/decide, etc.).
- **Dataset en informe:** describir pipeline `prepare_dataset.py` y generación de `bench_*.json`.

Referencias internas

Recurso	Contenido
<code>informe/informe_tecnico.md §6</code>	Resultados y análisis (redactado)
<code>informe/notas_experimentos.md</code>	Observaciones cualitativas y métricas por bloque
<code>results/summary_bench.md</code>	Tabla agregada
<code>results/comparison_bench.png</code>	Gráfico <code>objective_score</code> por instancia
<code>execute.md</code>	Plan manual de ejecución (completado)
<code>planning/plan_trabajo.md</code>	Estado del proyecto
<code>planning/walkthroughD.md</code>	Diseño experimental Paso D
<code>planning/walkthroughE.md</code>	Solver unificado y enrutado